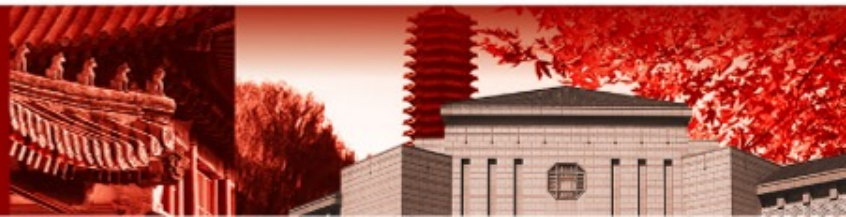


习题课-第13周



北京大学



目录

- 第 7- 9 次上机作业
- 第四次书面作业
- 补充内容



北京大学



上机作业：判断等价关系是否成立

- 题目：给定若干个包含变量的等式与不等式，形如 $x==y$, $x!=y$ 。判断在这些等式与不等式的约束下是否存在解
- 思路：
 - 无解的情况意味着，某个不等式约束与构建的等价类矛盾了
 - 只需要先遍历所有等式，利用并查集构建所有等价类
 - 然后遍历所有不等式，判断不等式是否与所有等式约束相容
- 注意：
 - 需要先接收全部的输入，先遍历所有等式，后遍历所有不等式
 - 按照输入的顺序，不区分等式与不等式是有问题的。
 - 对于一个较早出现的不等式，尽管它可能与之前的所有等式相容，但它仍可能与后续的等式矛盾



北京大学



上机作业：Eqs

- 题目：给定整数方程，其中变量的取值范围为 $[-50, 50]$ ，求全部解的数量。

$$a_1x_1^3 + a_2x_2^3 + a_3x_3^3 + a_4x_4^3 + a_5x_5^3 = 0$$

- 直接的思路：枚举遍历
 - 对于任意的一组输入 a_i ，判断一组 x_i 取值是否构成解代价为 $O(1)$
 - 但是总的枚举数量为 $101^5 > 10^9$ ，超时
- 优化枚举过程中的操作
 - 例如，在前四层遍历下，可以直接计算 x_5 的取值，优化掉一层遍历
 - 即，计算出 $a_5x_5^3$ 的取值，从而判断是否存在解



北京大学

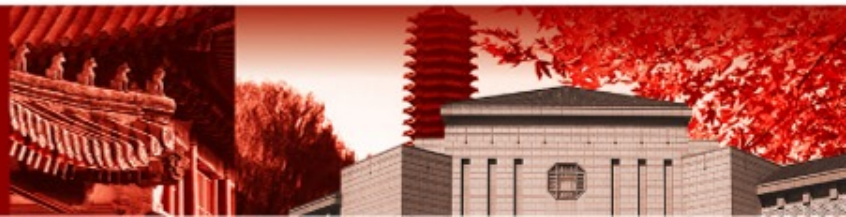


上机作业：Eqs

- 优化枚举过程中的操作
 - 进一步，可以尝试优化两层遍历，只遍历前三个变量
 - 固定 x_1, x_2, x_3 ，可以直接计算出 $a_4x_4^3 + a_5x_5^3$ 的取值
 - 如何快速通过这一取值来判断是否 (x_4, x_5) 有解？
- 利用哈希检索
 - 将 (x_4, x_5) 视为关键字， $a_4x_4^3 + a_5x_5^3$ 视为散列函数的一部分
 - 由于限定了取值范围，关键字的数量是有限的，共 101^2 个
 - 选取适当的散列函数，例如 $(a_4x_4^3 + a_5x_5^3) \% 1001$ ，或利用 Python 内置的字典



北京大学



上机作业：Eqs

- 空间换时间的思想：
 - 利用哈希表，将五层遍历优化为三层遍历，代价为需要额外存储 101^2 个关键字
 - 用内存空间，换取运行时间
 - 本质在于，哈希表在大多数情况下，能够做到以 $O(1)$ 的复杂度进行检索、插入、删除，就是利用了空间换时间的思想
- 空间与时间的权衡
 - 本题中，若只使用两层遍历，就需要额外存储 $101^3 > 1\text{ M}$ 个关键字
 - 空间的上限为 $65536\text{ KB} \approx 65\text{ MB}$ ，而 Python 中一个整数占用的空间不少于 24 B （随整数值动态变化）
 - 可能导致 MLE



北京大学



上机作业：成绩排序

- 题目：给出班里某门课程的成绩单，请你按成绩从高到低对成绩单排序输出，如果有相同分数则名字字典序小的在前。
- 涉及多个排序码的排序问题
- 方法一：使用稳定的排序算法，进行两次排序。
 - 先按优先级低的排序码排序（名字，字典序升序）
 - 然后使用稳定的排序算法，按优先级高的排序码排序（成绩，降序）



北京大学



上机作业：成绩排序

- 方法二：将其视为复合的排序码，自定义比较规则
 - 例如，定义 Student 类，并实现该类的 `__lt__` 方法，定义比较规则
 - 存储 Student 对象列表，调用 `sort` 时将使用该方法进行元素间的比较

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def __lt__(self, other):
        if self.score == other.score:
            return self.name > other.name
        else:
            return self.score < other.score

students = []
for _ in range(n):
    name, score = input().split()
    score = int(score)
    students.append(Student(name, score))

students.sort(reverse=True)
```



北京大学



上机作业：成绩排序

- 更方便的实现：利用 Python 内置的 sort 函数
 - 利用 sort 函数对复合数据类型的排序行为
 - 如果待排序列表中的元素都是列表，或元组
 - sort 函数排序时，会先比较第一个元素、再比较第二个元素
 - 每一个元素的排序次序都是默认的升序排序
- 本题中，需要对成绩降序排序，对名字升序排序
 - sort 无法对每个元素分别定义排序次序
 - 但可以利用 key 参数来定义排序码
 - 将排序码定义为（成绩的相反数，名字字符串）元组，按这一排序码进行升序排序，就实现了目的



北京大学



上机作业：成绩排序

- sort 函数的 key 参数
 - key 参数接受一个函数，用于计算自定义的排序码
 - 如果不使用该参数，排序码就是元素本身；
 - 如果使用该参数，sort 函数对于每个元素先计算出排序码，再按所有排序码排序
 - 具体来说，key 函数接受一个元组 $x = (\text{grade}, \text{name})$ ，返回 $(-\text{grade}, \text{name})$ 即可
- 对于这类简单的函数，可以使用 Python 的 lambda 表达式
 - lambda 表达式用于定义匿名函数，语法为 `lambda arg : expression`
 - 例如，`lambda x : (-x[0], x[1])` 就实现了上述功能

```
for _ in range(n):  
    name, score = input().split()  
    students.append((int(score), name))  
  
students.sort(key=lambda x: (-x[0], x[1]))
```

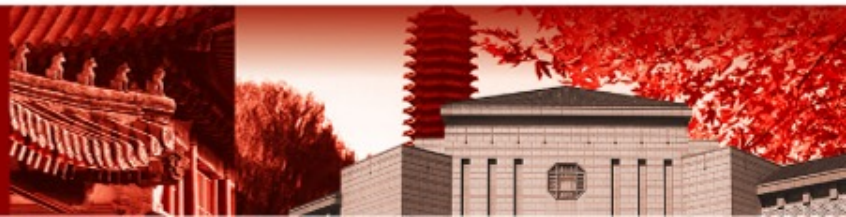


上机作业：距离排序

- 题目：给出三维空间中的 n 个点（不超过10个）,求出 n 个点两两之间的距离,并按距离由大到小依次输出两个点的坐标及它们之间的距离，保留两位小数。
- 对于距离相同的点对，要求先输出靠前的点对
 - 按出现顺序构建点对列表，然后使用稳定排序算法即可
 - 或者，可以将距离、出现次数视为两个排序码，方法同前一题
- 注意：
 - 浮点数的比较不能用 `==`
 - 保留两位小数在输出时使用格式化输出即可。不能在排序前就进行截断，可能导致错误

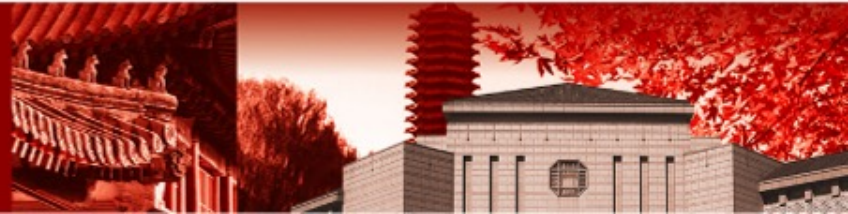


北京大学



Python 中的 sort / sorted

- 用法：假设 x 为待排序的列表 / 元组
 - `x.sort()`
 - `x = sorted(x)`
- `sort / sorted` 使用的排序算法为 TimSort 算法
 - 是插入排序与归并排序的结合
 - 是稳定的排序算法
 - 最好情况下的复杂度为 $O(n)$ ，最坏情况下的复杂度为 $O(n \log n)$
- TimSort 的算法设计本身非常复杂
 - 参考 <https://zhuanlan.zhihu.com/p/695042849>
 - 显然，不在考试范围中
 - Python 内置的所有库与函数，在上机考试中可以直接调用

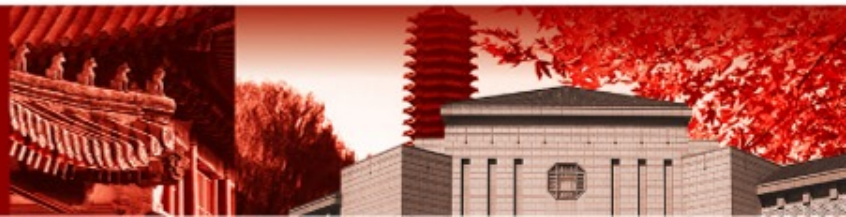


上机作业：求逆序对数

- 题目：对于一个长度为 N 的整数序列 A ，满足 $i < j$ 且 $A_i > A_j$ 的数对 (i, j) 称为整数序列 A 的一个逆序对。请求出整数序列 A 的所有逆序对个数。
- 归并排序的经典例题
 - 归并排序：将序列平均分为两段，先排序第一段，再排序第二段，然后将两个有序序列合并为一个有序序列
 - 计数逆序对，也可以使用分治法：
 - 先计算第一段序列的逆序对
 - 再计算第二段序列的逆序对
 - 然后计算两个元素分别位于第一段和第二段的逆序对
 - 关键主要在第三步。如果两个序列都是有序的，计算起来就很简单
 - 基于归并排序改进即可



北京大学

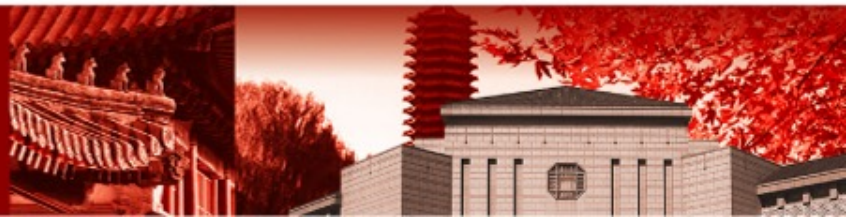


上机作业：求逆序对数

- 题目：对于一个长度为 N 的整数序列 A ，满足 $i < j$ 且 $A_i > A_j$ 的数对 (i, j) 称为整数序列 A 的一个逆序对。请求出整数序列 A 的所有逆序对个数。
- 归并排序的经典例题
 - 归并排序：将序列平均分为两段，先排序第一段，再排序第二段，然后将两个有序序列合并为一个有序序列
 - 计数逆序对，也可以使用分治法：
 - 先计算第一段序列的逆序对
 - 再计算第二段序列的逆序对
 - 然后计算两个元素分别位于第一段和第二段的逆序对
 - 关键主要在第三步。如果两个序列都是有序的，计算起来就很简单
 - 基于归并排序改进即可



北京大学

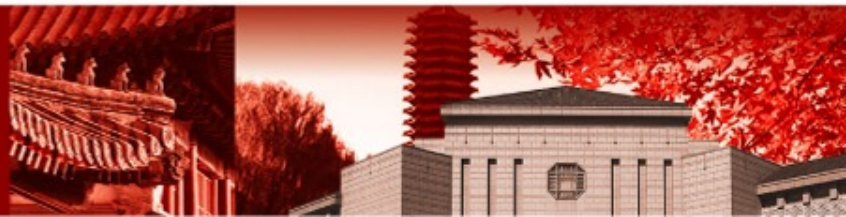


上机作业：求逆序对数

```
def merge_sort_and_count(num_list, start, end):  
    result = 0  
    if start + 1 >= end:  
        return result  
  
    mid = (start + end - 1) // 2  
  
    result += merge_sort_and_count(num_list, start, mid + 1)  
    result += merge_sort_and_count(num_list, mid + 1, end)  
    result += merge_and_count(num_list, start, mid, end)  
    return result
```



北京大学



上机作业：求逆序对数

```
def merge_and_count(num_list, start, mid, end):
```

```
    temp_list = []
```

```
    left, right = start, mid + 1
```

```
    result = 0
```

```
    while left < mid + 1 and right < end:
```

```
        if num_list[left] <= num_list[right]:
```

```
            temp_list.append(num_list[left])
```

```
            left += 1
```

```
        else:
```

```
            temp_list.append(num_list[right])
```

```
            right += 1
```

```
            result += mid + 1 - left
```

```
    while left < mid + 1:
```

```
        temp_list.append(num_list[left])
```

```
        left += 1
```

```
    while right < end:
```

```
        temp_list.append(num_list[right])
```

```
        right += 1
```

```
    for i in range(len(temp_list)):
```

```
        num_list[start + i] = temp_list[i]
```

```
    return result
```



北京大学



补充题目：交换次数

- 给定一个序列，要求仅通过交换相邻两个元素的操作，对序列进行排序。求最少进行的交换次数
- 冒泡排序就是这样一种排序方法，实现一遍冒泡排序，并记录次数即可
 - 不过，时间复杂度为 $O(n^2)$
 - 另一个问题：如何说明冒泡排序的交换次数是最小的
- 利用逆序对来进行分析
 - 交换相邻两个元素，可能导致逆序对数减少一、增加一，或者不变
 - 假定原逆序对数为 n ，则至少进行 n 次相邻交换
 - 冒泡排序中，每一次交换，都使逆序对数减少一
 - 明白这一点后，求出原序列的逆序对数即可，复杂度为 $O(n \log n)$



课堂练习

- 再来看 (C) 题

- 如果执行归并排序，应该递归调用 MergeSort([Q, H, C, Y, P, A]) 和 MergeSort([M, S, R, D, F, X])，直到最低一层
- 一次/一趟扫描，应理解为对整个序列进行合并
- 实际上与答案 5 不符，该序列为直接将相邻元素两两归并得到的序列

- 在内排序的过程中，采取不同排序方法会产生不同的排序中间结果。设要将序列{**Q**, H, C, Y, **P**, A, M, S, **R**, D, F, X}按字母序的升序重新排列，则

- (A) 是冒泡排序的一次扫描结果，
- (B) 是初始步长为4的希尔排序的一次扫描结果，
- (C) 是二路归并排序的一次扫描结果，
- (D) 是以第一个元素为分界元素的快速排序的一次扫描结果

- 供选择的答案 A~E:

- 1, F, H, C, D, P, A, M, Q, R, S, Y, X
- 2, P, A, C, S, Q, D, F, X, R, H, M, Y
- 3, A, D, C, R, F, Q, M, S, Y, P, H, X
- 4, H, C, Q, P, A, M, S, R, D, F, X, Y
- 5, H, Q, C, Y, A, P, M, S, D, R, F, X



北



课堂练习

- 另一道题：对一组记录进行降序排序，其关键码为（46, 70, 56, 38, 40, 80, 60, 22），写出归并排序第一轮归并结果
- 结果为（46, 70, 38, 56, 40, 80, 22, 60）
 - 恰好为相邻元素两两合并产生的序列
 - 其原因在于，总元素数量恰好为 2 的幂



北京大学



书面作业

- 对序列 8, 3, 9, 2, 1, 7, 5, 10, 6, 4 升序排列，使用二路归并排序，最后一次归并之前的结果
 - 正确答案：[1, 2, 3, 8, 9], [4, 5, 6, 7, 10]
 - 许多同学采用了相邻元素两两归并的方式
 - 最后得到 [1, 2, 3, 5, 7, 8, 9, 10], [6, 4]
 - 严格来说是错误的，因为归并排序不会产生这一结果
- 使用希尔排序，初始步长为 4，写出第一趟排序后的结果
 - 第一趟，同一趟扫描一样，是指每一分组都排序，不是只排一个分组
 - 分组为：[8, 1, 6], [3, 7, 4], [9, 5], [2, 10]
 - 排序后的分组：[1, 6, 8], [3, 4, 7], [5, 9], [2, 10]
 - 第一趟排序后的序列：[1, 3, 5, 2, 6, 4, 9, 10, 8, 7]



北京大学



书面作业

- 判断题：由于碰撞的发生，基于散列表的检索仍然需要进行关键码对比，并且关键码的比较次数仅取决于选择的散列函数与处理碰撞的方法两个因素。
- 应该判断为错误，因为影响比较次数的重要因素是负载因子
 - 问题：只要确定了散列函数和处理碰撞方法，每个元素的检索过程是否就是确定的？
 - 并没有完全确定，一个元素检索过程的比较次数，还依赖于表中所有元素的分布情况
 - 微观层面，针对特定元素如此；在宏观层面，元素的平均比较次数就依赖于负载因子



北京大学



书面作业

- 判断题：数据表中有 10000 个元素，如果仅要求求出其中最大的 10 个元素，则适合使用堆排序算法。
- 应该判断为正确，堆数据结构适用于取出最小/最大元素的操作。对于该问题，只需要以代价 $O(n)$ 建堆，再执行 10 次代价为 $O(\log n)$ 的 extract-min 操作即可
 - 问题：遍历 10 次，或者说利用选择排序，排出前 10 个元素，同样只需要 $O(n)$ 代价。
 - 这种做法的复杂度不低于堆排序，不能够否定原题
 - 更严谨一些，题目要求求出前 k 大的元素，且 k 远小于 n
 - 则两种做法的复杂度分别为 $O(n+k\log n)$ 、 $O(kn)$



北京大学



书面作业

- 哈希表的长度为11，槽编号从0开始。哈希函数为 $h(\text{key}) = \text{key} \% 11$ ，使用线性探查法解决冲突。向空表中依次插入 0, 22, 12, 16, 23。假设每个元素的查找概率相同，求查找成功时的平均查找长度。

0	1	2	3	4	5	6	7	8	9	10
0	22	12	23		16					

- 注意，计算ASL时，需要先计算关键码的基地址
- 不要把“连续的元素段长度”当作比较次数，误写成 $(1+2+3+4+1) / 5$
- 正确答案: $(1+2+2+3+1) / 5 = 1.8$



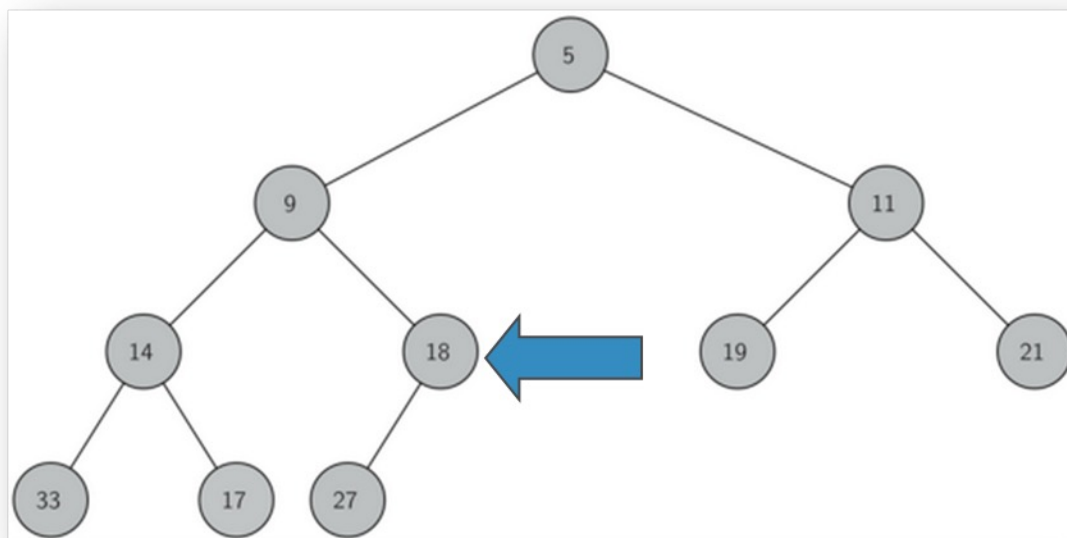
北京大学



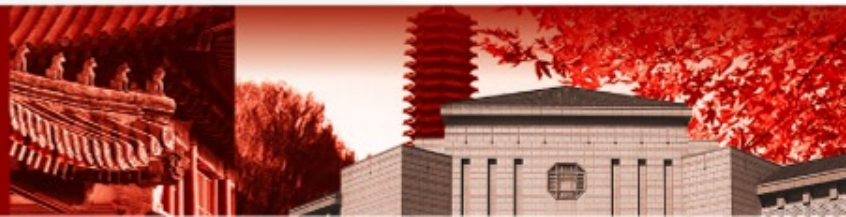
堆的操作总结

- 建堆过程

- 从最后一个非叶结点开始，依次向前调整
- 每次调整结点 v ，使得以 v 为根的子树满足堆性质
- 调整方式为 sift down，不断将该结点与子结点比较与交换



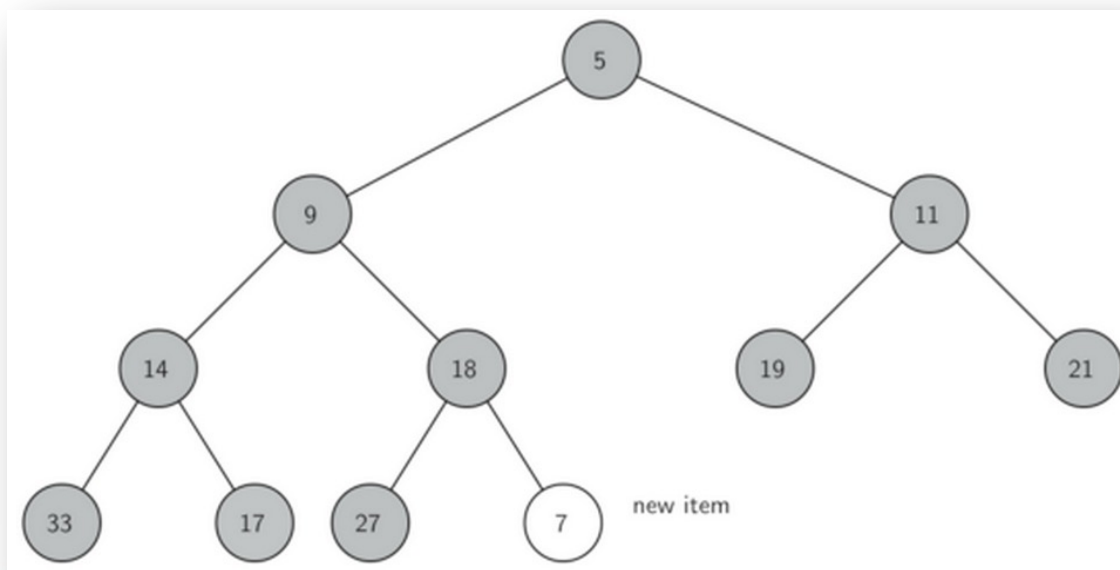
北京大学



堆的操作总结

- 插入过程

- 先将新元素插入至末尾
- 这一插入过程可能破坏其所在子树的堆性质
- 进行 sift up 操作，不断将该结点与其父结点比较和交换

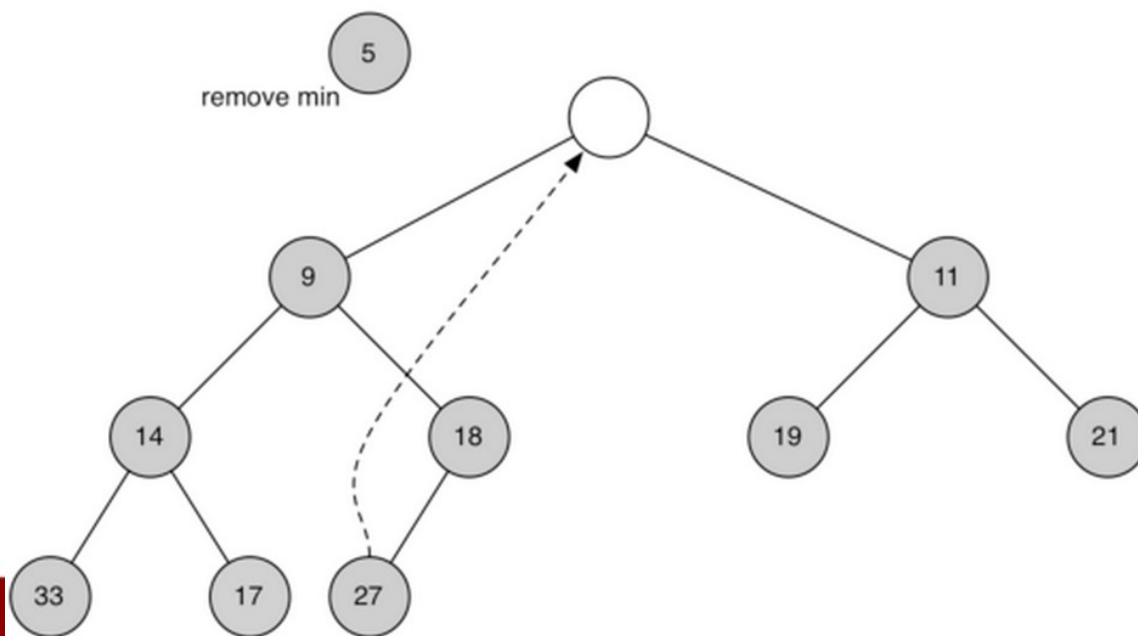


北京大学



堆的操作总结

- 删除过程（取出最小元素）
 - 用最后一个结点替代根节点
 - 这一过程同样破坏了堆性质
 - 对根节点进行 sift down 调整，不断与子结点比较与交换



北京大学

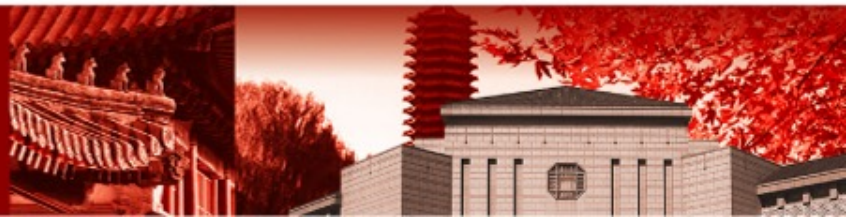


堆的操作总结

- 插入与删除操作的时间复杂度都为 $O(\log n)$
 - 调整的次数最大为树的高度，即 $O(\log n)$
- 建堆操作的时间复杂度为 $O(n)$
 - 共进行了 $O(n)$ 次调整，每次调整的时间复杂度为 $O(\log n)$
 - 但由此得出的 $O(n \log n)$ 并不是一个紧的渐进上界



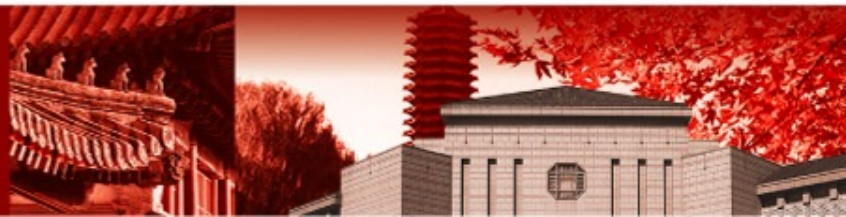
北京大学



堆的操作总结

- 为计算方便，假设建堆结束后，最后一层恰好填满
 - 树高为 h ，根结点位于第 0 层，叶结点位于第 h 层
 - 总结点数为 $n = 2^{h+1} - 1$ ， $h = \log(n + 1) - 1$
 - 第 i 层 ($0 \leq i < h$)，包含 2^i 个结点，sift down 调整的次数最多为 $h - i$
 - 总代价为：

$$\begin{aligned} \sum_{i=0}^{h-1} (h-i) \times 2^i &= \sum_{j=1}^h j \times 2^{h-j} = \sum_{j=1}^h j \times 2^{\log(n+1)-1-j} \times 2^{-j} = O(n) \times \sum_{j=1}^h \frac{j}{2^j} \\ &= O(n) \end{aligned}$$



补充内容

- 两个整数集合 S_1 、 S_2 ，大小分别为 N 、 M ， $M = O(\log N)$ ，试借助于排序来求 S_1 和 S_2 的交集。请给出算法思想、伪代码描述和时间复杂度分析。



北京大学

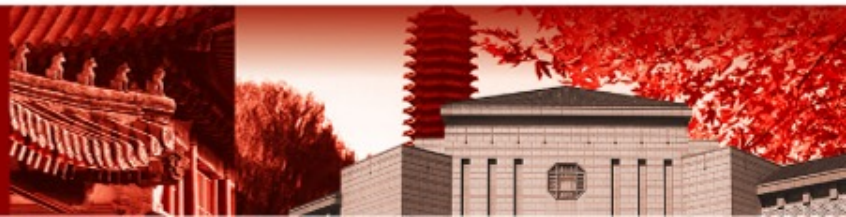


补充内容

- 两个整数集合 S_1 、 S_2 ，大小分别为 N 、 M ， $M = O(\log N)$ ，试借助于排序来求 S_1 和 S_2 的交集。请给出算法思想、伪代码描述和时间复杂度分析。
- 算法(1)：只对 S_1 进行排序，然后遍历 S_2 ，在 S_1 中使用二分查找来判断 S_2 中的元素是否也在 S_1 中出现。时间复杂度： $N * \log N$ （对 S_1 排序）+ $M * \log N$ （二分查找），即 $O(N \log N)$ 。



北京大学



补充内容

- 两个整数集合 S_1 、 S_2 ，大小分别为 N 、 M ， $M = O(\log N)$ ，试借助于排序来求 S_1 和 S_2 的交集。请给出算法思想、伪代码描述和时间复杂度分析。
- 算法(2)：对 S_1 、 S_2 分别排序，然后遍历 S_2 ，对于 S_2 的每一个元素 $S_2[i]$ ，在 S_1 中使用二分查找来找到大于等于 $S_2[i]$ 的第一个元素 $S_1[j]$ ，若 $S_2[i] = S_1[j]$ ，则 $S_2[i]$ 放入结果集合。由于 S_2 也已排好序，下一次二分查找的初始下界为 $j + 1$ 。时间复杂度分析与算法(1)类似。



北京大学

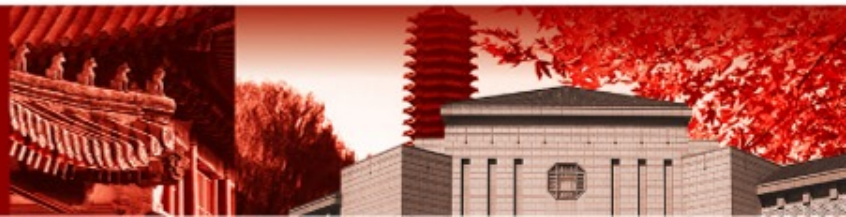


补充内容

- 两个整数集合 S_1 、 S_2 ，大小分别为 N 、 M ， $M = O(\log N)$ ，试借助于排序来求 S_1 和 S_2 的交集。请给出算法思想、伪代码描述和时间复杂度分析。
- 算法(3)：对 S_1 、 S_2 分别排序，然后使用归并排序的思路求 S_1 、 S_2 的交集。时间复杂度分析： $N \log N$ （对 S_1 排序）
+ $M \log M$ （对 S_2 排序）+ $N + M$ （归并），即 $O(N \log N)$ 。



北京大学

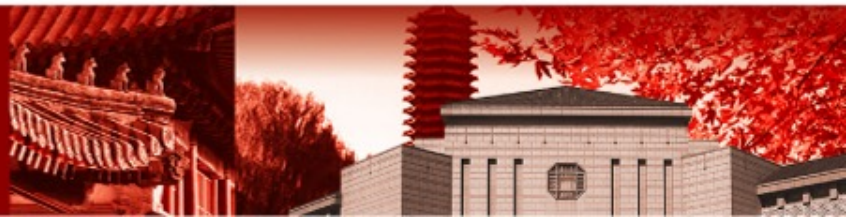


补充内容

- 如何用基数排序的思想，对一系列单词集合进行字典序排序，使得复杂度为 $O(\text{所有单词长度之和})$ 。



北京大学



补充内容

- 如何用基数排序的思想，对一系列单词集合进行字典序排序，使得复杂度为 $O(\text{所有单词长度之和})$ 。

首先，按照字符串的长度进行保序的桶排序，使得字符串由短到长的排列。该过程时间复杂度为 $O(m + l_{\max})$ ，其中 $l_{\max} = \max\{l_i\}$ 。注意到：

$$m + l_{\max} \leq m + \sum_{i=1}^m l_i - (m - 1) = O\left(\sum_{i=1}^m l_i\right)$$

然后，进行 $l_{\max}$ 次桶排序。第 i 次排序只对长度大于等于 $l_{\max} - i + 1$ 的字符串的第 $l_{\max} - i + 1$ 位进行桶排序。并用计数变量 k 去记录满足条件的字符串的下标的范围 $[k, \dots, m]$ ，使得 $l_k \geq l_{\max} - i + 1$ 并且 $l_{k-1} < l_{\max} - i + 1$

假设长大于等于 l 的字符串数目为 $n_{\geq l}$ 。则排序的时间代价为

$$O\left(\sum_{l=1}^{l_{\max}} (n_{\geq l} + 26)\right) = O\left(\sum_{i=1}^m l_i\right)$$

因而总的排序时间复杂度为 $O(\sum_{i=1}^m l_i)$



北京大学



补充内容

- 若长度为 n ($n > 1$) 的两个字符串A和B满足: $A=B$ 或者 存在 $k \in (0, n)$ 使得,

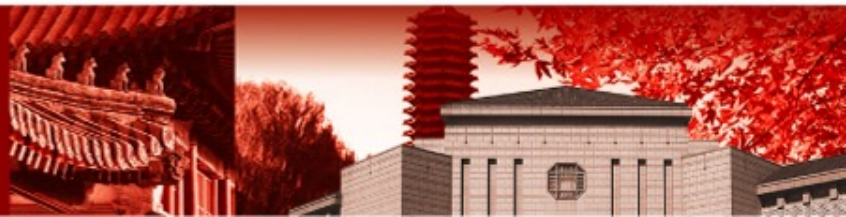
a) $A[0] = B[k]$ ↵

b) $A[1 \dots (n - k - 1)] = B[(k + 1) \dots (n - 1)]$ ↵

c) $A[(n - k) \dots (n - 1)] = B[0 \dots (k - 1)]$ ↵

•

- 则称A和B互相为彼此的循环旋转。例如, arc 和 car 是彼此的循环旋转。请设计一个**线性时间**的算法, 判断字符串A是否是另一个字符串B的循环旋转, 并简要说明正确性。



补充内容

- 生成新字符串 $T=A+A$ ，在 T 中用 KMP 方法查找 B 是否存在。若存在，则 A 和 B 互相为彼此的循环旋转，若不存在，则不是。
- arccarc 中含有 arc 的全部循环旋转。

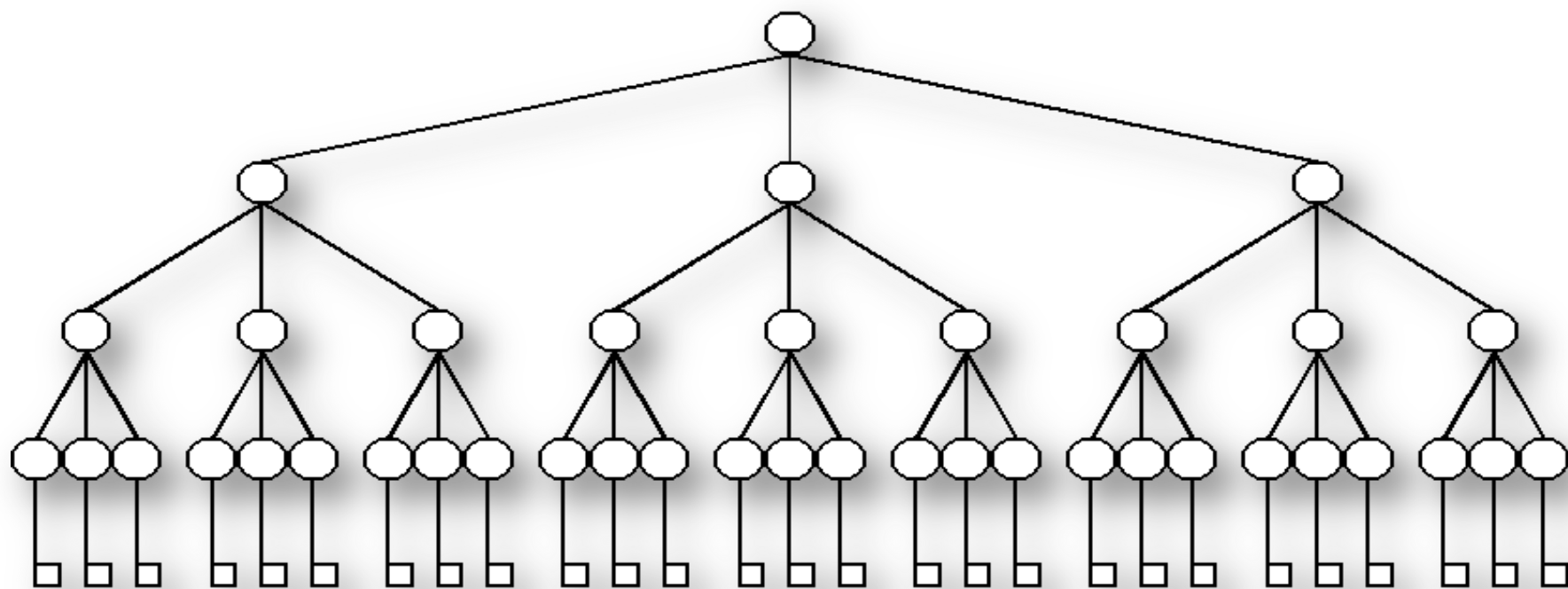


北京大学



多路归并树

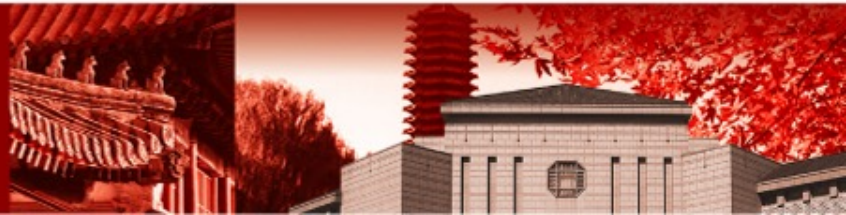
- k 路归并指每次将 k 个顺串合并成一个顺串



多(3)路归并



北京大学



多路归并树分析

- 在做 k 路归并的时候，每次都需要做 $k-1$ 次比较才能找出所要的记录，代价较大
- 选择树
 - ➡ 赢者树
 - ➡ 败者树
- 目的
 - ➡ 提高在 k 个归并串的当前值中找到最小值的效率



北京大学

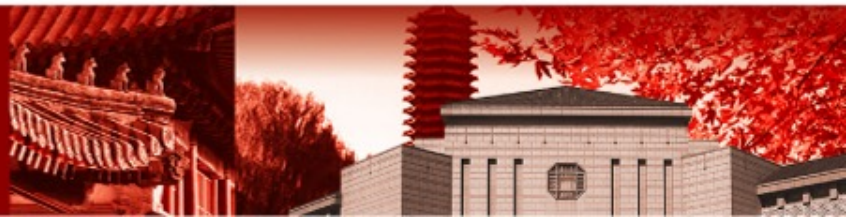


赢者树

- 用完全二叉树作为存储结构
 - 叶结点用数组 $L[1..n]$ 表示，内部结点用数组 $B[1..n-1]$ 表示
 - 数组 B 中实际存放的是数组 L 的索引

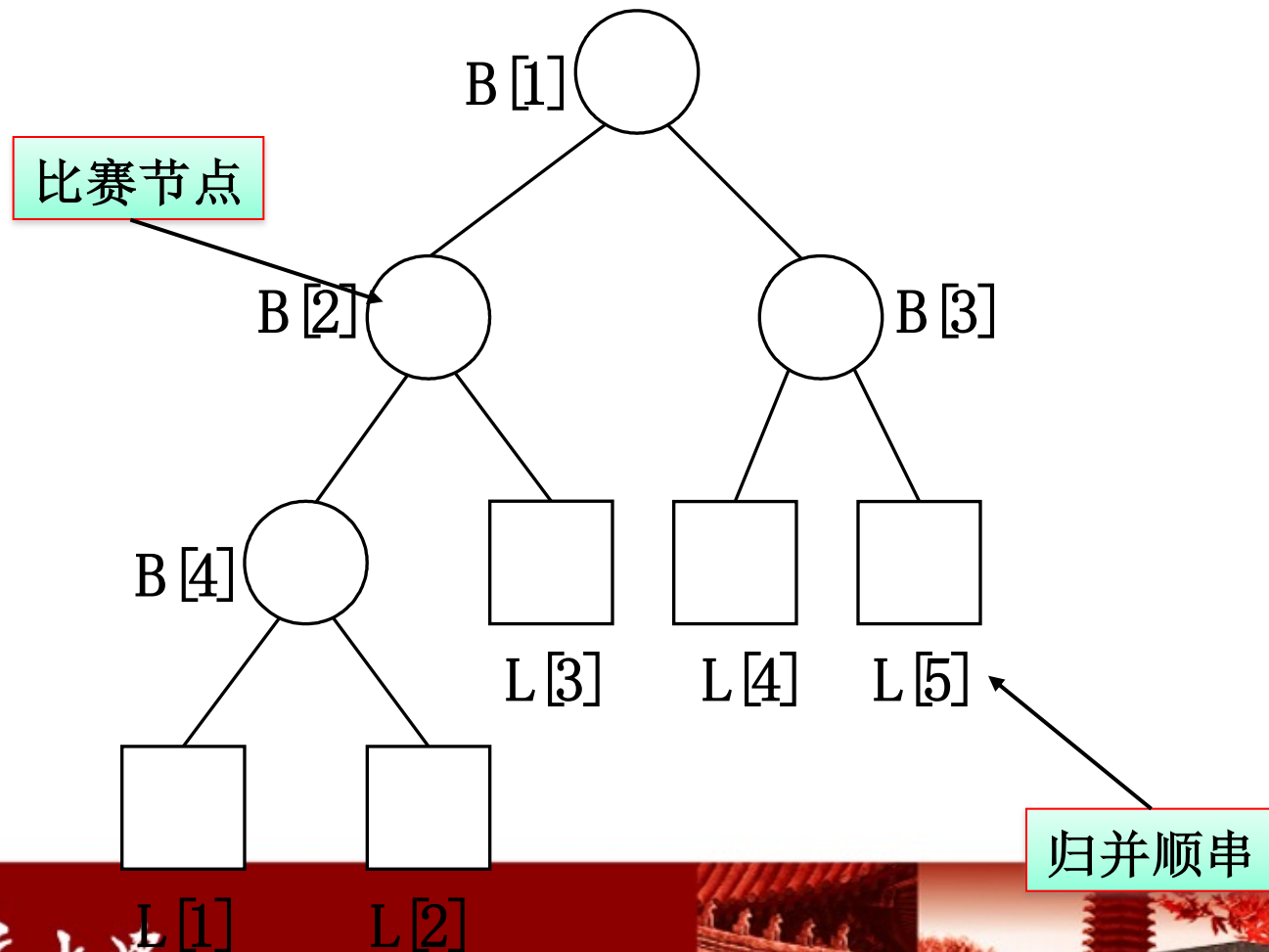


北京大学



赢者树的结构

n 路归并，赢者树具有 $2n-1$ 个节点



北京大学

外部结点L[i]与内部父结点B[p]关系

- 外部结点数为 n ，内部节点数为 $n-1$ ，比赛树深度 $s = \lceil \log_2 n \rceil - 1$

—如右图，比赛树（只考虑内部节点）深度为 $s = \lceil \log_2 5 \rceil - 1 = 2$

- 最底层最左端内部结点编号为 2^s

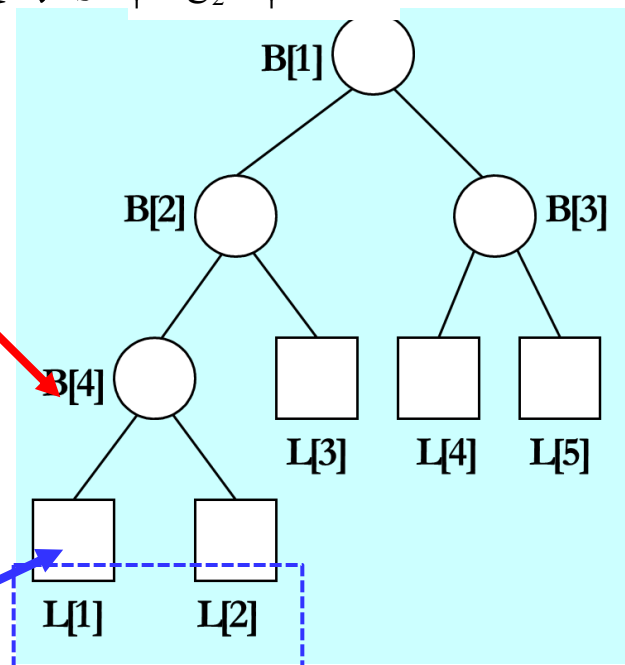
—如右图， $B[4] = 2^2 = 4$

- 最底层的内部结点数为 $n - 2^s$

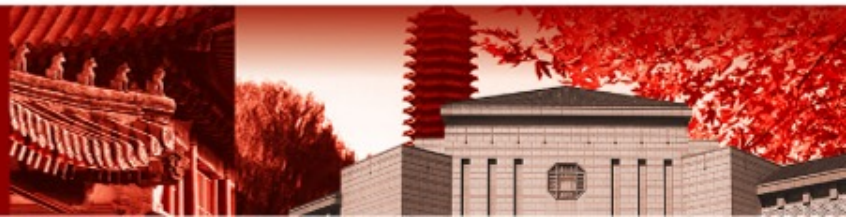
—如右图， $n - 2^s = 5 - 4 = 1$

- 最底层的外部结点数（LowExt）为最底层内部结点数的2倍

—即： $\text{LowExt} = 2 * (n - 2^s) = 2 * 1 = 2$



北京大学



外部结点L[i]与内部父结点B[p]关系

- 最底层外部结点之上的所有(内+外)

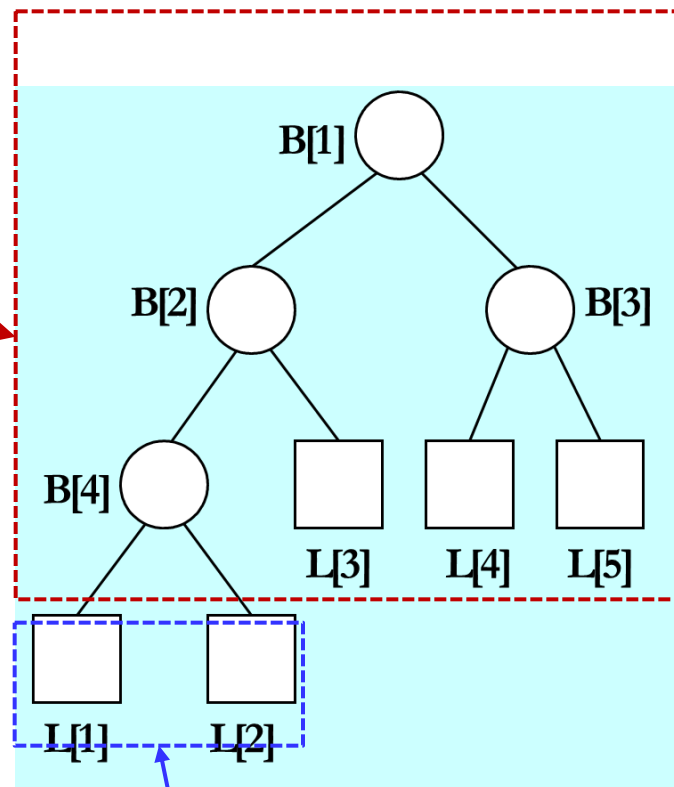
结点数目 (offset) , $offset =$

$$2^{(s+1)} - 1$$

—如右图, $offset = 2^{(2+1)} - 1 = 7$

- 外部结点L[i]与内部父结点B[p]关系可以表示如下:

$$p = \begin{cases} (i + offset) / 2, & i \leq LowExt \\ (i - LowExt + n - 1) / 2, & i > LowExt \end{cases}$$



北京大学



赢者树的特点

- 通过比较两个选手的分数确定一场比赛的赢家
 - 从树最底层开始，每两个树叶之间进行比赛，输的选手被淘汰，赢的继续向上进行竞赛，树根记录了整个比赛的胜者。
- 如果选手 $L[i]$ 的分值改变，可以修改这棵赢者树
 - 沿着从 $L[i]$ 到根结点的路径，和兄弟节点的值进行比较，根据比赛结果修改二叉树节点的值（赢者上），而不必改变树其他部分的比赛结果

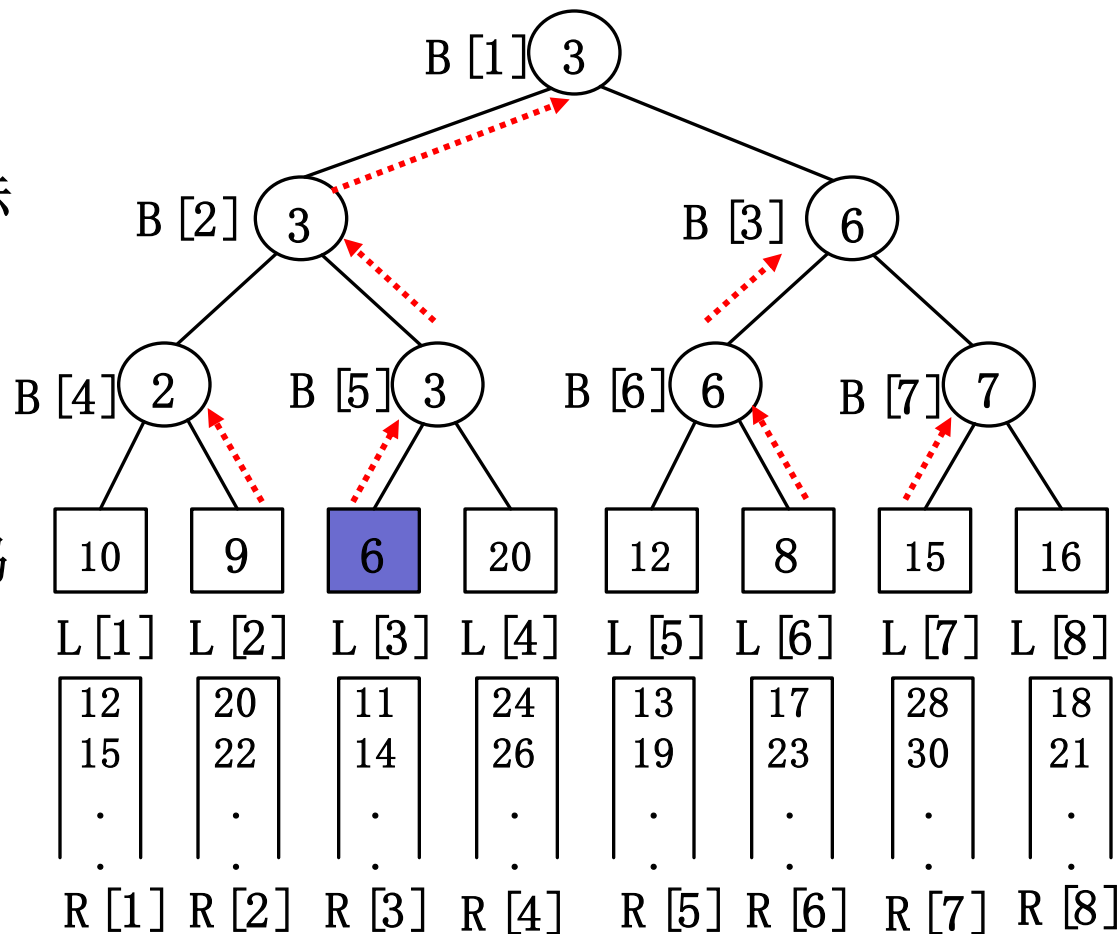


北京大学



赢者树的示例

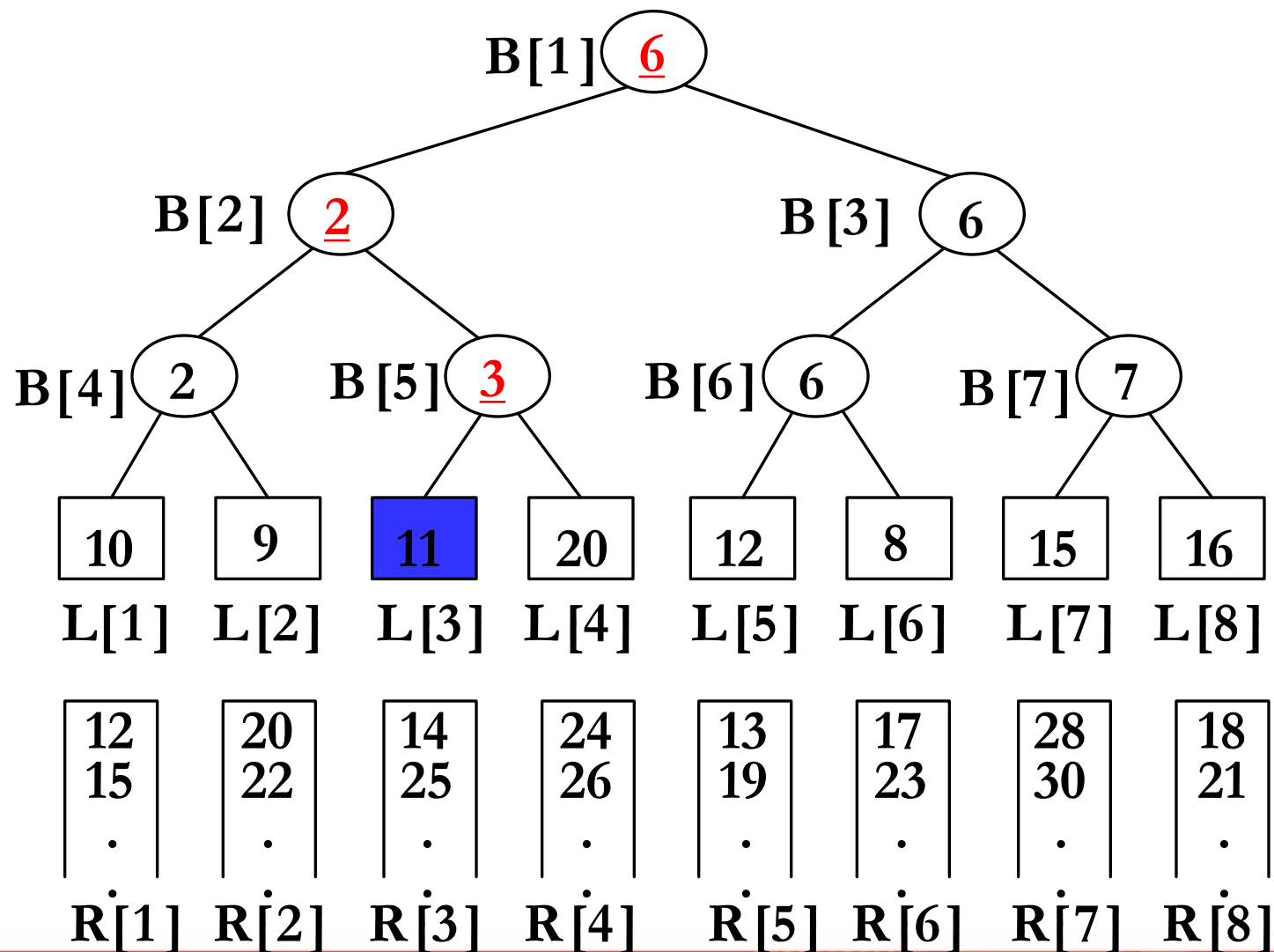
数组B中存储的是L的下标



北京大学



赢者树的重构



北京大学



败者树

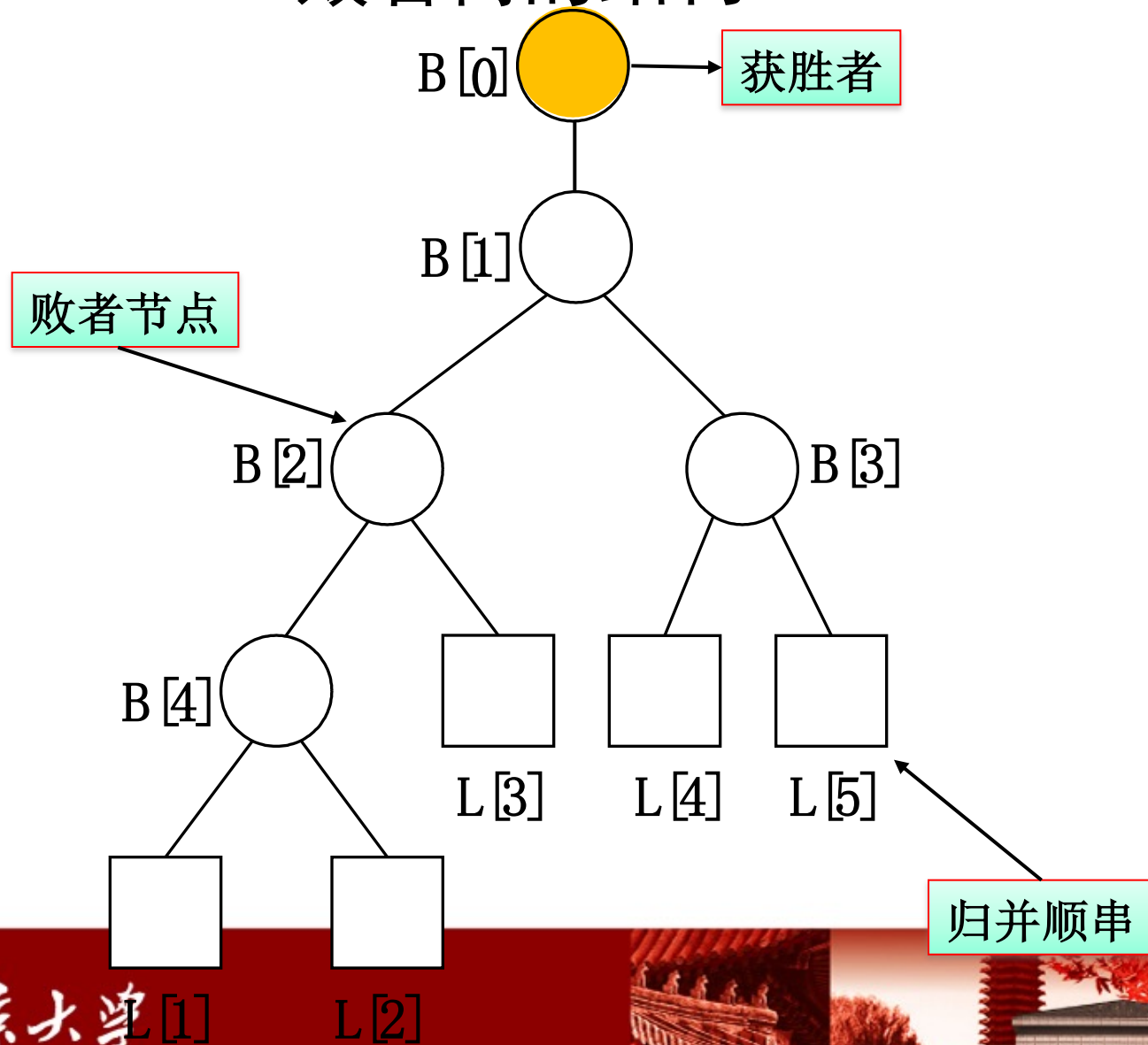
- 是赢者树的一种变体。
- 在败者树中，用父结点记录其左右子结点进行比赛的败者，而让获胜者去参加更高阶段的比赛。
- 新增根结点 $B[0]$ ，来记录整个比赛的最终胜者。
- 败者树是为了简化重构过程，树结构未变。



北京大学



败者树的结构



北京大学

$L[1]$

$L[2]$



比赛过程

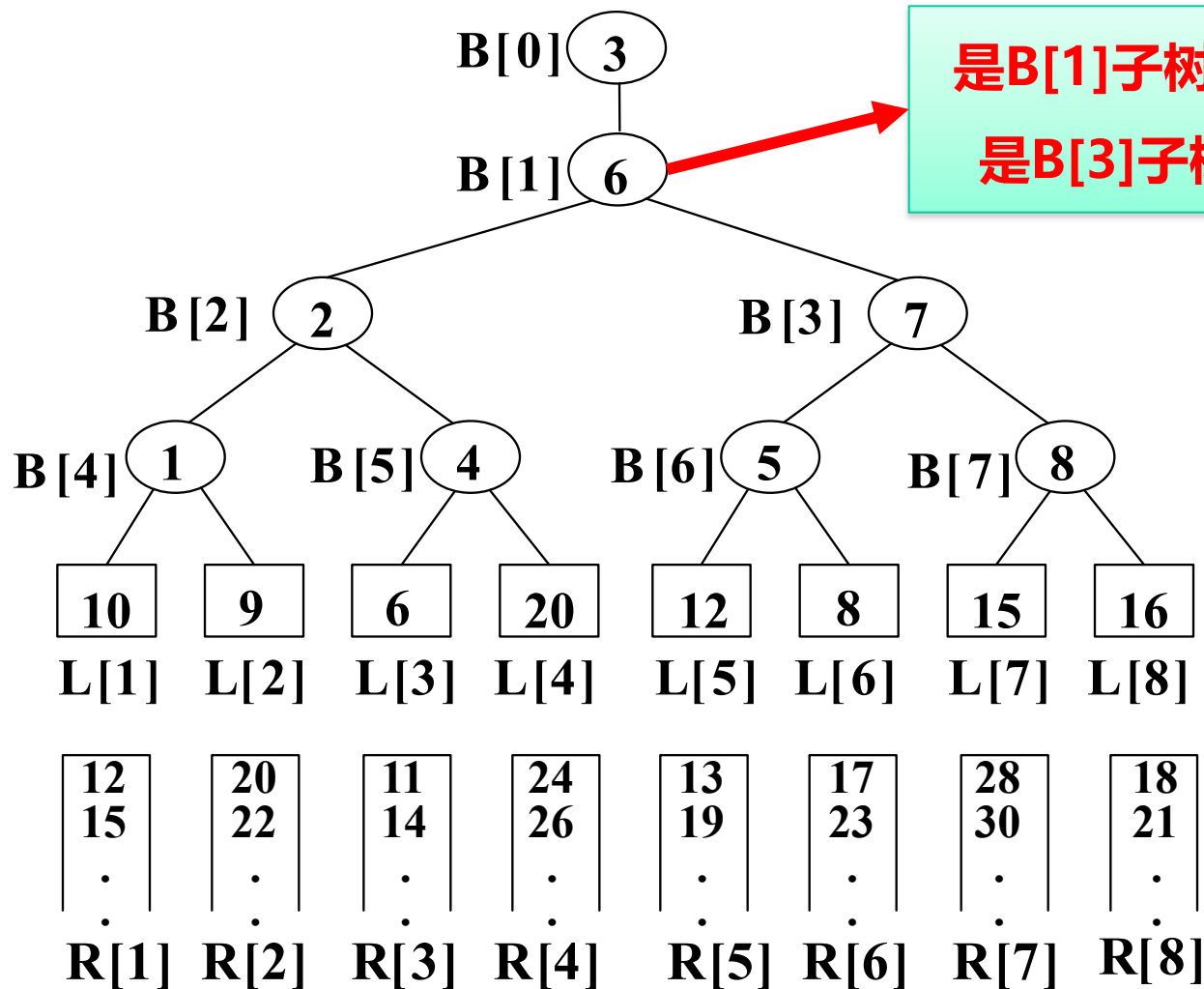
- 将新进入树的结点与其父结点进行比赛
 - 把败者存放在父结点中
 - 而把胜者再与上一级的父结点进行比赛
- 这样的比赛不断进行，直到结点B[1]
 - 把败者的索引放在结点B[1]
 - 把胜者的索引放到结点B[0]



北京大学



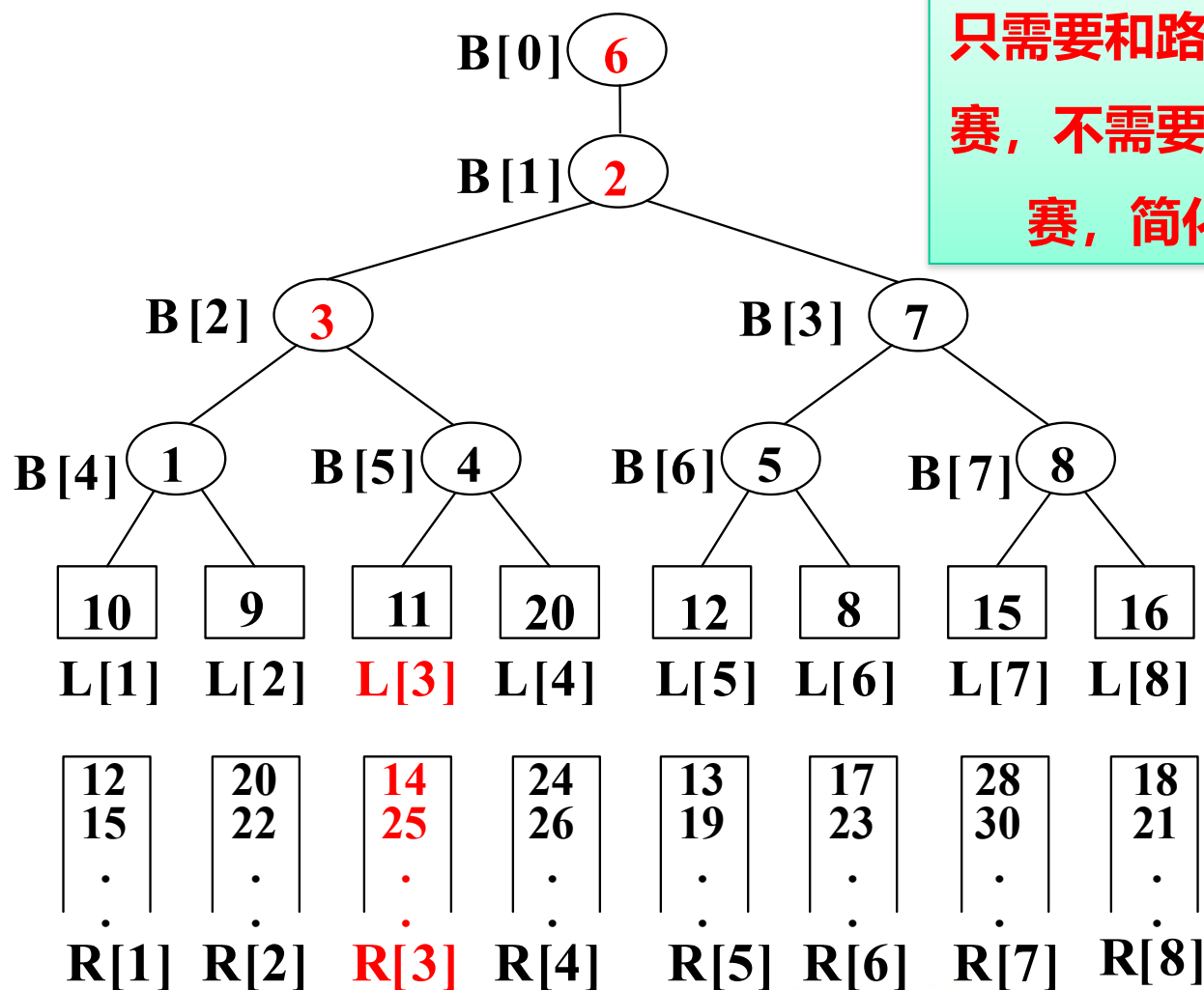
败者树的示例



北京大学



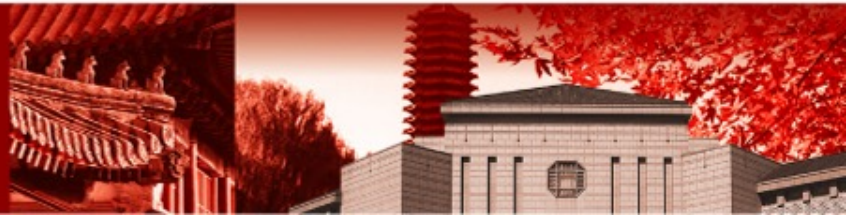
败者树的重构



只需要和路径上的节点比赛，不需要和兄弟节点比赛，简化重构过程



北京大学



存在重复元素

给你一个整数数组 `nums` 。如果任一值在数组中出现 至少两次 ，返回 `true` ；如果数组中每个元素互不相同，返回 `false` 。

示例 1:

输入: `nums = [1,2,3,1]`

输出: `true`

解释:

元素 1 在下标 0 和 3 出现。

示例 2:

输入: `nums = [1,2,3,4]`

输出: `false`

解释:

所有元素都不同。

示例 3:

输入: `nums = [1,1,1,3,3,4,3,2,4,2]`

输出: `true`



北京大学



存在重复元素

- 先排序，然后判断相邻的两个元素是否相等，如果相等则说明有重复的元素。



北京大学



存在重复元素

- 创建一个哈希表，**key**为遍历数组中的数字，**value**为出现次数，最后判断出现次数即可。



北京大学



存在重复元素

- 直接比较nums列表和nums集合的长度。



北京大学



H指数

给你一个整数数组 `citations`，其中 `citations[i]` 表示研究者的第 `i` 篇论文被引用的次数。计算并返回该研究者的 `h` 指数。

根据维基百科上 [h 指数的定义](#)：`h` 代表“高引用次数”，一名科研人员的 `h` 指数是指他（她）至少发表了 `h` 篇论文，并且至少有 `h` 篇论文被引用次数大于等于 `h`。如果 `h` 有多种可能的值，`h` 指数是其中最大的那个。

示例 1：

输入：`citations = [3,0,6,1,5]`

输出：3

解释：给定数组表示研究者总共有 5 篇论文，每篇论文相应的被引用了 3, 0, 6, 1, 5 次。

由于研究者有 3 篇论文每篇至少被引用了 3 次，其余两篇论文每篇被引用不多于 3 次，所以她的 `h` 指数是 3。

示例 2：

输入：`citations = [1,3,1]`

输出：1



北京大学



H指数

首先我们可以将初始的 H 指数 h 设为 0，然后将引用次数排序，并且对排序后的数组从大到小遍历。

根据 H 指数的定义，如果当前 H 指数为 h 并且在遍历过程中找到当前值 $citations[i] > h$ ，则说明我们找到了一篇被引用了至少 $h + 1$ 次的论文，所以将现有的 h 值加 1。继续遍历直到 h 无法继续增大。最后返回 h 作为最终答案。



北京大学



H指数

```
sorted_citation = sorted(citations, reverse = True)
h = 0; i = 0; n = len(citations)
while i < n and sorted_citation[i] > h:
    h += 1
    i += 1
return h
```

复杂度分析

- 时间复杂度： $O(n \log n)$ ，其中 n 为数组 *citations* 的长度。即为排序的时间复杂度。
- 空间复杂度： $O(\log n)$ ，其中 n 为数组 *citations* 的长度。即为排序的空间复杂度。



北京大学



H指数

根据上述解法我们发现，最终的时间复杂度与排序算法的时间复杂度有关，所以我们可以使用计数排序算法，新建并维护一个数组 *counter* 用来记录当前引用次数的论文有几篇。

根据定义，我们可以发现 H 指数不可能大于总的论文发表数，所以对于引用次数超过论文发表数的情况，我们可以将其按照总的论文发表数来计算即可。这样我们可以限制参与排序的数的大小为 $[0, n]$ （其中 n 为总的论文发表数），使得计数排序的时间复杂度降低到 $O(n)$ 。

最后我们可以从后向前遍历数组 *counter*，对于每个 $0 \leq i \leq n$ ，在数组 *counter* 中得到大于或等于当前引用次数 i 的总论文数。当我们找到一个 H 指数时跳出循环，并返回结果。



北京大学



H指数

```
n = len(citations); tot = 0
counter = [0] * (n+1)
for c in citations:
    if c >= n:
        counter[n] += 1
    else:
        counter[c] += 1
for i in range(n, -1, -1):
    tot += counter[i]
    if tot >= i:
        return i
return 0
```

- 时间复杂度： $O(n)$ ，其中 n 为数组 *citations* 的长度。需要遍历数组 *citations* 一次，以及遍历长度为 $n + 1$ 的数组 *counter* 一次。
- 空间复杂度： $O(n)$ ，其中 n 为数组 *citations* 的长度。需要创建长度为 $n + 1$ 的数组 *counter*。



北京大学



H指数

我们需要找到一个值 h ，它是满足「有 h 篇论文的引用次数至少为 h 」的最大值。小于等于 h 的所有值 x 都满足这个性质，而大于 h 的值都不满足这个性质。同时因为我们可以用较短时间（扫描一遍数组的时间复杂度为 $O(n)$ ，其中 n 为数组 *citations* 的长度）来判断 x 是否满足这个性质，所以这个问题可以用二分搜索来解决。

设查找范围的初始左边界 *left* 为 0，初始右边界 *right* 为 n 。每次在查找范围内取中点 *mid*，同时扫描整个数组，判断是否至少有 mid 个数大于 mid 。如果有，说明要寻找的 h 在搜索区间的右边，反之则在左边。



北京大学



H指数

我们需要找到一个值 h ，它是满足「有 h 篇论文的引用次数至少为 h 」的最大值。小于等于 h 的所有值 x 都满足这个性质，而大于 h 的值都不满足这个性质。同时因为我们可以用较短时间（扫描一遍数组的时间复杂度为 $O(n)$ ，其中 n 为数组 *citations* 的长度）来判断 x 是否满足这个性质，所以这个问题可以用二分搜索来解决。

设查找范围的初始左边界 *left* 为 0，初始右边界 *right* 为 n 。每次在查找范围内取中点 *mid*，同时扫描整个数组，判断是否至少有 *mid* 个数大于 *mid*。如果有，说明要寻找的 h 在搜索区间的右边，反之则在左边。

```
left, right = 0, len(citations)
while left < right:
    # +1 防止死循环
    mid = (left + right + 1) >> 1
    cnt = 0
    for v in citations:
        if v >= mid:
            cnt += 1
    if cnt >= mid:
        # 要找的答案在 [mid, right] 区间内
        left = mid
    else:
        # 要找的答案在 [0, mid) 区间内
        right = mid - 1
return left
```

复杂度分析

- 时间复杂度： $O(n \log n)$ ，其中 n 为数组 *citations* 的长度。需要进行 $\log n$ 次二分搜索，每次二分搜索需要遍历数组 *citations* 一次。
- 空间复杂度： $O(1)$ ，只需要常数个变量来进行二分搜索。

用最少数量的箭射爆气球

有一些球形气球贴在一堵用 XY 平面表示的墙面上。墙面上的气球记录在整数数组 `points`，其中 `points[i] = [x_start, x_end]` 表示水平直径在 `x_start` 和 `x_end` 之间的气球。你不知道气球的确切 y 坐标。

一支弓箭可以沿着 x 轴从不同点 完全垂直 地射出。在坐标 `x` 处射出一支箭，若有一个气球的直径的开始和结束坐标为 `x_start`, `x_end`，且满足 `x_start ≤ x ≤ x_end`，则该气球会被 引爆。可以射出的弓箭的数量 没有限制。弓箭一旦被射出之后，可以无限地前进。

给你一个数组 `points`，返回引爆所有气球所必须射出的 最小 弓箭数。

示例 1:

输入: `points = [[10,16],[2,8],[1,6],[7,12]]`

输出: 2

解释: 气球可以用2支箭来爆破:

-在 `x = 6` 处射出箭，击破气球 `[2,8]` 和 `[1,6]`。

-在 `x = 11` 处发射箭，击破气球 `[10,16]` 和 `[7,12]`。

示例 2:

输入: `points = [[1,2],[3,4],[5,6],[7,8]]`

输出: 4

解释: 每个气球需要射出一支箭，总共需要4支箭。

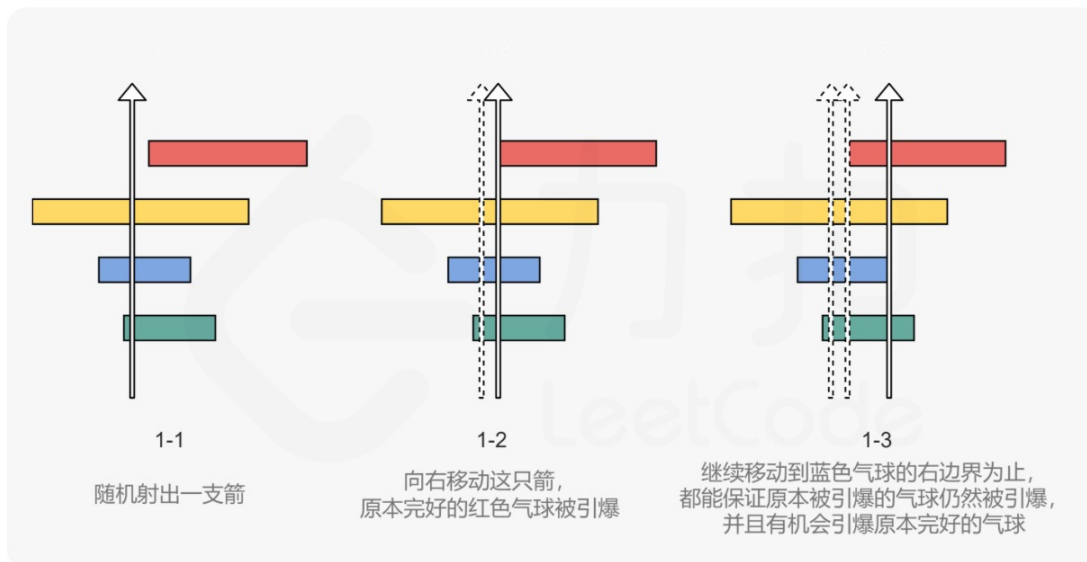


北京大学



用最少数量的箭射爆气球

我们首先随机地射出一支箭，再看一看是否能够调整这支箭地射出位置，使得我们可以引爆更多数目的气球。



如图 1-1 所示，我们随机射出一支箭，引爆了除红色气球以外的所有气球。我们称所有引爆的气球为「原本引爆的气球」，其余的气球为「原本完好的气球」。可以发现，如果我们将这支箭的射出位置稍微往右移动一点，那么我们就有机会引爆红色气球，如图 1-2 所示。



北京大学



用最少数量的箭射爆气球

那么我们最远可以将这支箭往右移动多远呢？我们唯一的要求就是：原本引爆的气球只要仍然被引爆就行了。这样一来，我们找出原本引爆的气球中右边界位置最靠左的那一个，将这支箭的射出位置移动到这个右边界位置，这也是最远可以往右移动到的位置：如图 1-3 所示，只要我们再往右移动一点点，这个气球就无法被引爆了。

为什么「原本引爆的气球仍然被引爆」是唯一的要求？别急，往下看就能看到其精妙所在。

因此，我们可以断定：

一定存在一种最优（射出的箭数最小）的方法，使得每一支箭的射出位置都恰好对应着某一个气球的右边界。

这是为什么？我们考虑任意一种最优的方法，对于其中的任意一支箭，我们都通过上面描述的方法，将这支箭的位置移动到它对应的「原本引爆的气球中最靠左的右边界位置」，那么这些原本引爆的气球仍然被引爆。这样一来，所有的气球仍然都会被引爆，并且每一支箭的射出位置都恰好位于某一个气球的右边界了。



北京大学

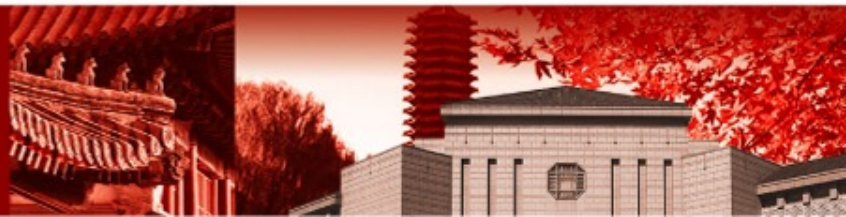


用最少数量的箭射爆气球

有了这样一个有用的断定，我们就可以快速得到一种最优的方法了。考虑**所有**气球中右边界位置最靠左的那一个，那么一定有一支箭的射出位置就是它的右边界（否则就没有箭可以将其引爆了）。当我们确定了一支箭之后，我们就可以将这支箭引爆的所有气球移除，并从剩下未被引爆的气球中，再选择右边界位置最靠左的那一个，确定下一支箭，直到所有的气球都被引爆。



北京大学



完全平方数

给你一个整数 n ，返回 和为 n 的完全平方数的最少数量。

完全平方数 是一个整数，其值等于另一个整数的平方；换句话说，其值等于一个整数自乘的积。例如，1、4、9 和 16 都是完全平方数，而 3 和 11 不是。

示例 1：

输入： $n = 12$

输出：3

解释： $12 = 4 + 4 + 4$

示例 2：

输入： $n = 13$

输出：2

解释： $13 = 4 + 9$



北京大学



完全平方数

动态规划的思想是，与当前状态对应的信息，可以由之前状态对应的信息递推出来。
对于此题来说，我们可以：

- 把一个数看作一个状态，记当前的数为 i ， i 之前的某个数为 x
- 把和为数 i 的完全平方数的最少数量看作状态对应的信息，记为 $dp[i]$

那现在问题就变成了，具体 i 之前的哪个 x 可以使得通过 $dp[x]$ 推出 $dp[i]$ ？

显然，在数 i 之前一定存在着许多数 x 使得 $x + \text{一个完全平方数} = i$ ，
即 $x + k * k = i (k = 1, 2, 3 \dots)$ ，也就是 x 与 i 之间差一个完全平方数，
即 $dp[x] + 1 = dp[i]$ 。

所以，我们只要找到所有与 i 差一个完全平方数的 x ，然后看这些 x 的哪个 $dp[x]$ 最小，把最小的 $dp[x] + 1$ ，就得到了 $dp[i]$ 。



北京大学



跳跃游戏

给你一个非负整数数组 `nums`，你最初位于数组的 第一个下标。数组中的每个元素代表你在该位置可以跳跃的最大长度。

判断你是否能够到达最后一个下标，如果可以，返回 `true`；否则，返回 `false`。

示例 1:

输入: `nums = [2,3,1,1,4]`

输出: `true`

解释: 可以先跳 1 步，从下标 0 到达下标 1，然后再从下标 1 跳 3 步到达最后一个下标。

示例 2:

输入: `nums = [3,2,1,0,4]`

输出: `false`

解释: 无论如何，总会到达下标为 3 的位置。但该下标的最大跳跃长度是 0，所以永远不可能到达最后一个下标。



北京大学



跳跃游戏

我们可以用贪心的方法解决这个问题。

设想一下，对于数组中的任意一个位置 y ，我们如何判断它是否可以到达？根据题目的描述，只要存在一个位置 x ，它本身可以到达，并且它跳跃的最大长度为 $x + \text{nums}[x]$ ，这个值大于等于 y ，即 $x + \text{nums}[x] \geq y$ ，那么位置 y 也可以到达。

换句话说，对于每一个可以到达的位置 x ，它使得 $x + 1, x + 2, \dots, x + \text{nums}[x]$ 这些连续的位置都可以到达。

这样以来，我们依次遍历数组中的每一个位置，并实时维护 **最远可以到达的位置**。对于当前遍历到的位置 x ，如果它在 **最远可以到达的位置** 的范围内，那么我们就可以从起点通过若干次跳跃到达该位置，因此我们可以用 $x + \text{nums}[x]$ 更新 **最远可以到达的位置**。

在遍历的过程中，如果 **最远可以到达的位置** 大于等于数组中的最后一个位置，那就说明最后一个位置可达，我们就可以直接返回 `True` 作为答案。反之，如果在遍历结束后，最后一个位置仍然不可达，我们就返回 `False` 作为答案。



北京大学



跳跃游戏

以题目中的示例一

```
[2, 3, 1, 1, 4]
```

为例：

- 我们一开始在位置 0，可以跳跃的最大长度为 2，因此最远可以到达的位置被更新为 2；
- 我们遍历到位置 1，由于 $1 \leq 2$ ，因此位置 1 可达。我们用 1 加上它可以跳跃的最大长度 3，将最远可以到达的位置更新为 4。由于 4 大于等于最后一个位置 4，因此我们直接返回 `True`。



北京大学



跳跃游戏

我们再来看看题目中的示例二

```
[3, 2, 1, 0, 4]
```



- 我们一开始在位置 0，可以跳跃的最大长度为 3，因此最远可以到达的位置被更新为 3；
- 我们遍历到位置 1，由于 $1 \leq 3$ ，因此位置 1 可达，加上它可以跳跃的最大长度 2 得到 3，没有超过最远可以到达的位置；
- 位置 2、位置 3 同理，最远可以到达的位置不会被更新；
- 我们遍历到位置 4，由于 $4 > 3$ ，因此位置 4 不可达，我们也就不考虑它可以跳跃的最大长度了。

在遍历完成之后，位置 4 仍然不可达，因此我们返回 `False`。



北京大学



跳跃游戏II

给定一个长度为 n 的 0 索引整数数组 `nums`。初始位置为 `nums[0]`。

每个元素 `nums[i]` 表示从索引 i 向后跳转的最大长度。换句话说，如果你在 `nums[i]` 处，你可以跳转到任意 `nums[i + j]` 处：

- $0 \leq j \leq \text{nums}[i]$
- $i + j < n$

返回到达 `nums[n - 1]` 的最小跳跃次数。生成的测试用例可以到达 `nums[n - 1]`。

示例 1:

输入: `nums = [2,3,1,1,4]`

输出: 2

解释: 跳到最后一个位置的最小跳跃数是 2。

从下标为 0 跳到下标为 1 的位置，跳 1 步，然后跳 3 步到达数组的最后一个位置。

示例 2:

输入: `nums = [2,3,0,1,4]`

输出: 2



北京大学



跳跃游戏II

我们的目标是到达数组的最后一个位置，因此我们可以考虑最后一步跳跃前所在的位置，该位置通过跳跃能够到达最后一个位置。

如果有多个位置通过跳跃都能够到达最后一个位置，那么我们应该如何进行选择呢？直观上来看，我们可以「贪心」地选择距离最后一个位置最远的那个位置，也就是对应下标最小的那个位置。因此，我们可以从左到右遍历数组，选择第一个满足要求的位置。

找到最后一步跳跃前所在的位置之后，我们继续贪心地寻找倒数第二步跳跃前所在的位置，以此类推，直到找到数组的开始位置。



北京大学

